

chrome extension 구동 방식 및 아이디어

1. 가장 먼저 Load unpacked 로 실제 Chrome에 올립니다

예를 들어 프로젝트가 이렇게 되어 있다고 하겠습니다.

```
arxiv-helper/  
├─ manifest.json  
├─ background.js  
├─ arxiv.js  
├─ arxiv.css  
└─ ...
```



Chrome에서

```
chrome://extensions
```



로 들어간 다음,

```
Developer mode ON  
→ Load unpacked  
→ arxiv-helper 폴더 선택
```



합니다.

Chrome 공식 문서에서도 개발 중인 Extension은 이 방식으로 직접 로드하도록 안내합니다.  Chrome for Devel... +1

여기서부터 이미 1차 검증이 됩니다.

▼ overall

arXiv 웹페이지



Content Script



Service Worker



Backend API



- DBLP Adapter
- OpenReview Adapter
- CVF Adapter
- ACL Adapter

- └ ...
- ▼
- Resolver
- ▼
- 정규화된 결과(JSON)
- ▼
- Service Worker
- ▼
- Content Script
- ▼
- arXiv 화면에 Badge/UI 표시

ArXiv ↔ Content Script ↔ Extension Service Worker ↔ Backend Server(만들어야 함)

Content Script ↔ Extension Service Worker _ 여기는 message passing

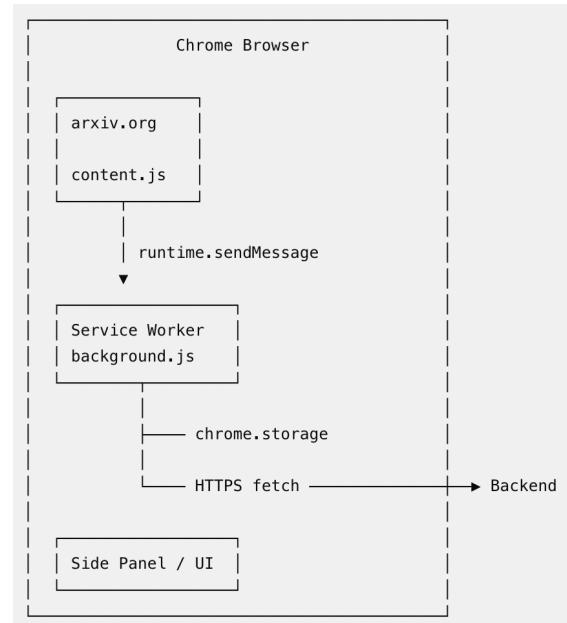
Extension Service Worker ↔ Backend Server _ 여기는 API request

Manifest V3 기준

service worker는 항상 켜져있는 서버가 아니다.

Interrupt처럼 신호를 받으면 wake up해서 작업 처리하고 chrome이 종료하는 순서로 작동

그 과정에서 가져가야할 데이터를 chrome storage나 backend DB에 저장해야한다.



용어	핵심 의미
Manifest V3	현재 Chrome Extension의 구조와 권한, 실행 방식을 정의하는 플랫폼 규격
manifest.json	Extension에서 어떤 파일을 실행하고 어떤 권한을 사용할지 정의하는 설정 파일

용어	핵심 의미
Content Script	arXiv 페이지 안에서 실행되어 제목·저자·DOI 등을 읽고 UI를 삽입하는 코드
DOM	HTML 페이지를 JavaScript가 읽을 수 있게 표현한 구조
Service Worker	Content Script의 요청을 받고 API 호출 등을 관리하는 Extension의 background controller
Message Passing	Content Script와 Service Worker가 데이터를 주고받는 Chrome 내부 통신 방법
API	다른 서비스의 데이터나 기능을 프로그램으로 요청하는 인터페이스
API V2	특정 API의 두 번째 버전. OpenReview API V2와 Manifest V3는 서로 관계없는 개념
Endpoint	<code>/resolve</code> , <code>/papers/...</code> 처럼 API에서 특정 기능을 제공하는 URL
JSON	Extension과 backend가 주고받을 데이터 형식
Backend	DBLP/OpenReview 조회, 논문 matching, 최종 판정 등을 수행하는 서버 측 프로그램
Adapter	DBLP, OpenReview, CVF처럼 서로 다른 형식의 데이터를 공통 형식으로 변환하는 모듈
Resolver	여러 출처를 종합하여 Venue/Track/Decision을 최종적으로 판단하는 logic
Schema	논문과 판정 결과를 어떤 데이터 구조로 저장할지 정한 규칙
Cache	이미 조회한 결과를 저장하여 외부 API를 반복 호출하지 않게 하는 기능
Database / SQL	결과를 장기 저장하는 장소 / 그 database를 조회하는 언어
Rate Limit	GitHub 등의 API를 일정 횟수 이상 호출하지 못하도록 하는 제한

Content Script의 역할

arXiv 웹페이지를 읽고 UI를 집어넣는 역할

arXiv 페이지를 열어봤을 때, content script에서

arXiv ID, Title, Authors, DOI, Comments, 현재 URL, PDF URL를 추출한다.

그러면 `chrome.runtime.sendMessage()` function에 이 정보들을 담아서 Service Worker에게 전달

Service Worker

Content Script에서 `chrome.runtime.sendMessage()`에 담은 정보를 보고

1. local cache 확인
2. cache가 유효하면 (cache hit) 바로 반환
3. 없으면 backend 호출
4. 결과 저장
5. Content Script에 반환